

信息安全科技创新课程项目

项目设计报告

LLM-VulnHub

面向 AI 大模型应用的漏洞库管理与智能分析系统

项目项	内容
课题名称	AI 大模型漏洞库管理系统
项目名称	LLM-VulnHub：面向 AI 大模型应用的漏洞库管理与智能分析系统
技术路线	Next.js + FastAPI + PostgreSQL + pgvector + LangGraph + Celery + Docker Compose
核心能力	实时动态搜集 AI 漏洞 + AI 标准化抽取 + 漏洞库管理 + RAG 智能问答

1. 项目背景与意义

随着大语言模型在智能问答、Agent、代码生成、办公自动化、企业知识库、智能客服等场景中的广泛应用，围绕大模型应用的新型安全风险也逐渐增多。与传统 Web 漏洞或系统漏洞相比，大模型安全风险不仅存在于代码层和网络层，还可能出现在 Prompt、工具调用、RAG 检索、插件权限、模型输出以及上下文管理等环节。

例如，Prompt Injection 可能诱导模型忽略系统提示词，RAG 知识库投毒可能影响模型回答结果，Agent 工具调用缺乏权限控制可能导致越权操作，插件供应链风险可能扩大攻击面，模型输出不安全也可能进一步诱发业务风险。这些问题具有来源分散、描述非结构化、分类体系尚不统一、人工分析成本高等特点。

传统漏洞库通常主要面向操作系统、Web 应用、中间件、网络设备等对象，能够记录漏洞编号、漏洞描述、影响版本和修复建议，但对 AI 大模型应用中的新型安全问题支持不足。AI 安全漏洞往往与具体业务流程、Prompt 设计、工具权限、知识库召回和模型网关策略相关，仅依靠普通关键词检索难以快速定位问题。

因此，本项目拟设计并实现一个面向 AI 大模型应用场景的漏洞库管理与智能分析系统。系统利用大模型对漏洞文本进行结构化抽取、风险分类、评分分析和修复建议生成，并结合向量检索技术实现基于漏洞库的智能问答，帮助用户更高效地管理、检索和分析 AI 大模型相关安全漏洞。

2. 项目目标与范围

本项目目标是构建一个具备完整交互能力的 AI 大模型漏洞库管理系统。系统不仅支持基本的漏洞增删改查，还能够利用大模型完成漏洞信息抽取、风险评分、智能问答和报告生成。项目按照“基础可交付 + 智能分析 + 工程化扩展”的思路组织功能。

目标类型	具体目标
基础目标	实现漏洞信息的录入、查询、编辑、删除、分类管理。
动态采集目标	利用采集任务从公开漏洞源、AI 框架公告、GitHub 安全公告、安全博客等渠道动态搜集 AI 漏洞。
智能分析目标	利用大模型从原始漏洞文本中抽取结构化字段，并按照统一字段格式入库。
安全分析目标	对漏洞进行类型识别、风险分级、评分和修复建议生成。
检索问答目标	基于漏洞库构建 RAG 问答能力，支持自然语言查询。
工程化目标	使用前后端分离、数据库、向量检索、异步任务和容器化部署。
展示目标	提供可交互 Web 页面、数据看板和漏洞分析报告生成功能。

2.1 MVP 功能边界

为避免项目范围过大导致难以按时完成，第一版系统应优先保证主流程闭环。MVP 版本需要支持：漏洞 CRUD、手动/URL 导入、AI 结构化抽取、标准化入库、风险评分、语义检索/RAG 问答、漏洞详情展示和基础数据看板。为了符合课题要求，“实时动态搜集 AI 漏洞”被设计为 P1 核心功能：通过 Celery Beat 定时任务或手动触发采集任务，从配置好的公开来源周期性抓取新增漏洞文本，再由 AI 自动判断相关性、抽取字段、去重并入库。

优先级	功能	说明
P0	漏洞管理	增删改查、搜索、筛选、详情页，保证系统基础可用。
P0	AI 结构化抽取	输入原始漏洞文本，输出标题、类型、影响组件、攻击方式、修复建议等字段。
P0	风险评分	采用规则评分 + 大模型解释，输出分数、等级和评分理由。
P1	RAG 问答	基于 pgvector 召回漏洞记录，再调用大模型生成回答。
P1	数据看板	展示漏洞总数、风险等级分布、漏洞类型分布、Top 组件等。
P1	动态采集	从公开漏洞源、AI 框架公告、GitHub 安全公告、安全博客/RSS 等渠道动态搜集 AI 漏洞文本。
P1	标准化入库	采集文本经 AI 相关性判断、结构化抽取、去重和风险评分后，按照固定字段格式写入漏洞库。
P2	异步任务	使用 Celery + Redis 管理采集、分析、Embedding 生成、失败重试等任务。

3. 项目名称、定位与使用对象

3.1 项目名称

项目名称暂定为：LLM-VulnHub：面向 AI 大模型应用的漏洞库管理与智能分析系统。该名称能够同时体现项目的安全方向、大模型方向和漏洞情报管理属性。

3.2 项目定位

本项目不是攻击工具，也不是单纯的漏洞 CRUD 管理系统，而是一个结合大模型能力的 AI 安全情报管理平台。系统的核心定位是：面向 AI 大模型应用安全场景，构建一个支持漏洞信息管理、AI 结构化抽取、风险分级、RAG 智能问答与自动报告生成的漏洞情报管理系统。

3.3 使用对象

用户角色	使用场景
安全学习者	学习 AI 大模型安全漏洞类型、攻击方式和防护措施。
安全运维人员	维护企业内部 AI 应用相关漏洞库，并快速分析风险。
开发人员	查询框架、插件、Agent 应用中的安全风险和修复建议。
课程答辩评审	查看系统功能、技术实现、工程化设计和创新点。

4. 需求分析

4.1 功能性需求

4.1.1 漏洞管理功能

系统应支持对漏洞信息进行基本管理，包括新增漏洞记录、修改漏洞信息、删除漏洞记录、查看漏洞详情、按关键词搜索漏洞、按风险等级/漏洞类型/影响组件/标签进行筛选，以及按风险评分和创建时间进行排序。

字段	说明
title	漏洞标题。
vuln_type	漏洞类型，例如 Prompt Injection、RAG 泄露、Agent 越权、供应链风险等。
severity	风险等级，低危/中危/高危/严重。
score	风险评分，0-100。
affected_component	影响组件，例如 LangChain、LLM Plugin、Agent Framework、RAG Pipeline。
description	漏洞描述。
attack_method	攻击方式。
impact	影响范围。
mitigation	修复建议。
source / reference_url	来源和参考链接。
tags / status	标签和处理状态。

4.1.2 AI 结构化抽取功能

用户可以输入一段原始漏洞文本，例如安全公告、博客内容、论文摘要或漏洞描述。系统调用大模型自动抽取结构化字段，并将抽取结果返回给前端供用户确认。为降低大模型幻觉风险，系统要求大模型无法判断的信息填入unknown，并允许用户在入库前人工修改。

输入示例：

某 Agent 框架在工具调用过程中未对工具返回内容进行隔离，攻击者可以构造恶意工具输出，诱导大模型忽略原有系统提示词并执行非授权操作，可能导致敏感信息泄露或越权调用外部接口。

输出示例：

```
{
  "title": "Agent 工具调用中的 Prompt Injection 漏洞",
  "vuln_type": "Prompt Injection",
  "severity": "高危",
  "affected_component": "Agent Tool Calling",
  "attack_method": "通过恶意工具返回内容注入隐藏指令",
  "impact": "可能导致敏感信息泄露、系统提示词绕过或越权操作",
  "mitigation": "对工具返回内容进行隔离处理，并增加权限校验和输出过滤机制"
}
```

4.1.3 AI 漏洞动态采集与标准化入库功能

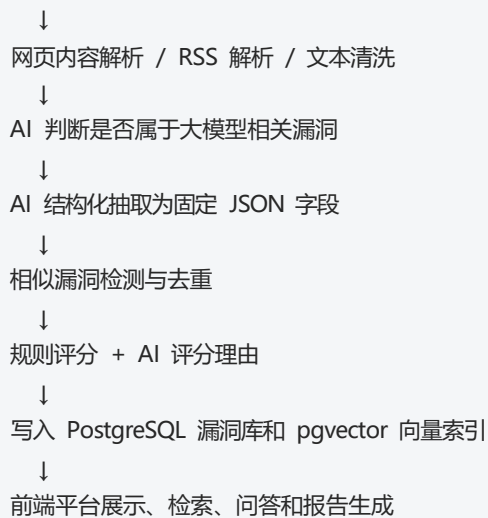
为了满足课题中“利用 AI 技术实时动态搜集 AI 漏洞，并按照固定格式上传到 AI 大模型漏洞系统平台”的要求，系统需要设计独立的动态采集模块。该模块允许用户在后台配置漏洞信息源，并通过定时任务周期性抓取新增漏洞文本。采集到的原始文本不会直接入库，而是先经过 AI 相关性判断、文本清洗、结构化抽取、相似去重和风险评估，最后按照系统固定字段格式写入漏洞库。

课程项目中可将“实时动态”实现为可配置周期的准实时采集：例如每 5 分钟、30 分钟或 1 小时执行一次采集任务。答辩演示时既可以手动触发采集，也可以展示后台任务自动发现新漏洞、抽取字段并入库的完整流程。

能力	设计说明
信息源配置	支持配置公开漏洞源、AI 框架安全公告、GitHub Security Advisory、RSS/安全博客、论文摘要或本地测试源。
动态采集	由 Celery Beat 周期性触发采集任务，也支持前端手动点击“立即采集”。
AI 相关性判断	大模型判断原始文本是否属于 AI/LLM/Agent/RAG 相关漏洞，非相关内容不进入漏洞库。
AI 结构化抽取	将原始文本转换为固定字段：标题、类型、影响组件、严重等级、攻击方式、影响范围、修复建议、来源链接等。
去重与合并	根据 URL、标题关键词和向量相似度检测重复漏洞，避免重复入库。
固定格式上传	通过后端入库接口将标准化结果写入 vulnerabilities 表，并生成 document_chunks 向量数据。
人工审核	对低置信度或字段缺失的结果标记为“待确认”，管理员确认后正式发布。

动态采集与入库流程：

```
公开漏洞源 / AI 框架公告 / GitHub 安全公告 / 安全博客 RSS
↓
Celery Beat 定时触发采集任务
```



4.1.4 风险评分功能

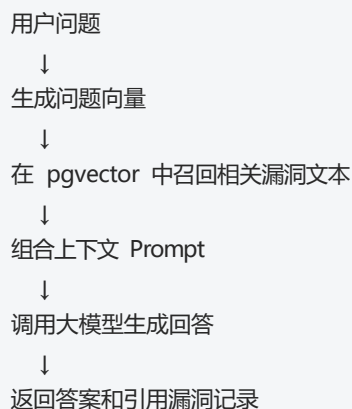
系统应根据漏洞字段自动计算风险评分。风险评分范围为 0 到 100，分数越高表示风险越大。为了避免完全依赖大模型导致评分不稳定，本项目采用“规则评分 + 大模型解释”的策略：规则系统负责计算分数，大模型负责解释评分原因和修复优先级。

评分因素	说明
漏洞严重性	严重、高危、中危、低危对应不同基础分。
是否可远程利用	能否远程触发，是否需要本地环境或特殊权限。
是否涉及敏感数据	是否可能泄露用户数据、密钥、系统提示词或知识库内容。
是否存在公开 PoC	是否有公开利用方式或可复现样例。
影响组件重要性	是否影响核心 Agent、RAG、插件、模型网关等关键组件。
修复复杂度	是否已有明确修复方案，修复是否会影响业务流程。
利用门槛	是否需要认证、特定权限或复杂攻击条件。

分数区间	风险等级
0 - 30	低危
31 - 60	中危
61 - 80	高危
81 - 100	严重

4.1.5 RAG 智能问答功能

系统应支持用户基于漏洞库进行自然语言问答。用户可以询问 Prompt Injection 的防护方式、RAG 知识库泄露风险、某个组件相关的高危漏洞等问题。系统先在漏洞库中召回相关记录，再将召回结果作为上下文提交给大模型生成回答。



4.1.6 相似漏洞检测功能

当用户录入新漏洞时，系统可以自动检索已有漏洞，判断是否存在相似记录。该功能可减少重复录入，提高漏洞库质量。实现上，新漏洞文本被转化为向量后，与已有漏洞向量进行相似度搜索，返回 Top-K 相似漏洞供用户确认。

4.1.7 报告生成功能

系统支持根据漏洞详情生成一份结构化分析报告。报告内容包括漏洞名称、漏洞类型、风险等级、风险评分、影响组件、漏洞描述、攻击方式、影响范围、修复建议、AI 分析结论和参考资料。MVP 阶段可以先实现 Markdown/页面展示，后续扩展 PDF 或 Word 导出。

4.1.8 数据看板功能

图表/指标	说明
漏洞总数	当前系统收录漏洞数量。
风险等级分布	低危、中危、高危、严重数量分布。
漏洞类型分布	Prompt Injection、RAG 泄露、供应链攻击等类型分布。
影响组件 Top-N	展示漏洞数量最多的框架、插件或组件。
最近新增漏洞	展示最新录入的漏洞。
高风险漏洞列表	展示风险评分最高的漏洞，方便答辩演示。

4.2 非功能性需求

类型	要求
可用性	前端界面清晰，用户能够完成从漏洞录入到 AI 分析、入库、检索问答的完整流程。
可维护性	后端模块分层清楚，便于后续扩展更多模型、更多漏洞来源和更多评分规则。
可扩展性	系统可扩展接入自动采集、多模型对比、报告导出和用户权限管理。

安全性	不在代码中保存敏感 API Key，后端接口进行参数校验和异常处理。
可解释性	AI 分析结果应保留评分理由、召回依据和生成过程日志。
稳定性	大模型调用失败时应返回明确错误信息，并支持重试或人工修改。
可部署性	使用 Docker Compose 一键启动前端、后端、数据库、Redis 等服务。

5. AI 漏洞实时动态搜集与标准化入库设计

本节专门说明系统如何实现“实时动态搜集 AI 漏洞，并按照固定格式上传到 AI 大模型漏洞系统平台”。系统将该能力拆分为信息源管理、采集任务调度、内容解析、AI 相关性判断、AI 结构化抽取、去重合并、标准化入库和审核发布八个环节。

5.1 信息源管理

系统提供信息源配置能力，每个信息源记录名称、类型、URL、采集周期、启用状态和最近采集时间。课程实现阶段可以优先支持 RSS/网页 URL/本地 JSON 三类来源，后续可扩展到 GitHub API、CVE/NVD API 或企业内部安全公告系统。

信息源类型	示例来源	采集方式
公开漏洞库	CVE/NVD、GitHub Security Advisory 等	API、RSS 或网页解析。
AI 框架公告	LangChain、LlamaIndex、Transformers、向量数据库等项目公告	GitHub Release、Issue、Advisory 或文档页面采集。
安全博客/RSS	安全研究团队博客、厂商安全公告、社区文章	RSS 订阅或网页正文抽取。
论文/报告摘要	大模型安全论文、技术报告、漏洞复现文章	手动导入或 URL 导入。
本地测试源	课程演示用 JSON/Markdown 文件	本地文件读取，保证无网络时也能演示。

5.2 采集任务调度

动态采集通过 Celery + Redis 执行，Celery Beat 负责周期性触发任务，Worker 负责抓取、解析和调用 AI 工作流。系统同时提供“立即采集”按钮，方便答辩演示。采集任务的执行状态、开始时间、结束时间、成功数量、失败数量和错误信息都会写入任务表，便于追踪和排错。

采集调度设计：

1. 管理员在前端配置数据源和采集周期。
2. Celery Beat 按周期创建 collect_source 任务。
3. Celery Worker 拉取任务并抓取新增内容。

4. 对每条候选文本调用 AI 标准化入库 workflow。
5. 写入采集日志、任务状态和错误信息。
6. 前端任务中心实时展示任务进度。

5.3 AI 相关性判断

公开信息源中并非所有内容都属于 AI 大模型漏洞，因此系统需要在入库前判断候选文本是否与 AI/LLM/Agent/RAG/模型供应链等主题相关。该步骤由关键词规则和大模型共同完成：规则先进行快速过滤，大模型再对候选文本进行语义判断，并输出置信度和判断理由。

相关性判断输出格式：

```
{
  "is_ai_vulnerability": true,
  "confidence": 0.91,
  "reason": "文本描述了 RAG 检索权限控制不足导致知识库泄露，属于 AI 应用安全漏洞。",
  "related_area": "RAG Data Leakage"
}
```

5.4 标准化抽取与固定格式上传

通过相关性判断的候选文本进入结构化抽取流程。系统要求大模型只输出符合 Schema 的 JSON 字段，随后使用 Pydantic 进行格式校验。如果 JSON 不合法或关键字段缺失，系统会自动重试一次；仍失败时将记录标记为“待人工处理”。校验通过后，后端调用统一入库服务写入漏洞表、分析记录表和向量 Chunk 表。

标准字段	来源
title	由 AI 根据正文和标题综合生成。
vuln_type	根据 OWASP LLM 风险类别和系统自定义类别归类。
affected_component	从公告中的框架、插件、模型服务、RAG 组件等信息抽取。
severity / score	严重等级由 AI 初判，分数由规则评分模块计算。
description	从原文摘要生成，保留事实性描述。
attack_method	抽取攻击路径、触发条件和利用方式。
impact	抽取可能造成的数据泄露、越权调用、拒绝服务等影响。
mitigation	抽取或生成修复建议，并标注是否来自原文。
source_url / collected_at	由采集器自动写入，保证可追溯。
status	高置信度设为已入库，低置信度设为待确认。

5.5 去重、合并与审核机制

动态采集容易产生重复内容，例如同一个漏洞可能同时出现在 GitHub Advisory、安全博客和漏洞库中。系统采用三层去重策略：首先根据 `source_url` 判断完全重复；其次根据标题、组件、漏洞类型做关键词匹配；最后使用 `pgvector` 进行相似度检索。如果相似度超过阈值，系统不直接新建漏洞，而是把新来源作为参考链接合并到已有漏洞，或将其标记为疑似重复等待人工确认。

去重策略：
URL 完全相同 → 跳过或更新已有记录
标题 + 组件 + 类型高度相似 → 合并来源
向量相似度 > 阈值 → 标记为疑似重复，进入人工审核
新漏洞且置信度高 → 标准化入库
新漏洞但字段缺失 → 待确认队列

5.6 课程实现范围

为了保证项目可完成，实时动态采集分为可演示版和增强版。可演示版优先实现本地测试源、RSS/网页 URL 导入、手动触发采集和定时任务；增强版再扩展 GitHub API、更多公开漏洞源和多源合并。这样既能满足课题要求，又能控制实现难度。

版本	实现内容
MVP 可演示版	本地 JSON/Markdown 测试源 + URL 导入 + 手动触发采集 + AI 标准化入库。
课程完整版	Celery Beat 定时采集 + 任务中心 + 去重合并 + 待确认审核队列。
增强版	接入 GitHub Security Advisory、CVE/NVD、更多安全博客 RSS，并支持多源情报合并。

6. 技术栈设计与选型依据

本项目采用现代 AI 应用工程技术栈，强调前后端分离、结构化数据与向量数据统一管理、AI 工作流可追踪和部署可复现。

层次	技术选型	作用
前端	Next.js + TypeScript	构建现代 Web 管理系统，提供页面路由、组件化开发和良好的工程结构。
UI	Tailwind CSS + shadcn/ui	快速构建简洁、美观、可复用的管理后台组件。
后端	FastAPI	提供 REST API、自动 OpenAPI 文档和高效的 Python AI 服务接口。
数据校验	Pydantic	对请求参数和响应数据进行类型校验，降低接口错误。

数据库	PostgreSQL	存储漏洞结构化数据、标签、分析记录 and 任务状态。
向量检索	pgvector	在 PostgreSQL 中存储和检索 embedding 向量，实现相似漏洞检索和 RAG。
ORM	SQLAlchemy 2.0	操作数据库并管理数据模型。
数据库迁移	Alembic	管理数据库表结构变更。
AI 工作流	LangGraph	编排漏洞抽取、分类、评分、报告生成等多步骤流程。
异步任务	Celery + Redis	执行 AI 分析、embedding 生成、定时采集等后台任务。
模型 API	DeepSeek API / OpenAI API / 本地模型	提供文本理解、抽取、生成和问答能力。
部署	Docker Compose	统一启动前端、后端、PostgreSQL、Redis 等服务。

6.1 选用 PostgreSQL + pgvector 的原因

漏洞库既需要结构化查询，也需要语义检索。结构化查询包括按漏洞类型、风险等级、影响组件、标签、时间和分数筛选；语义检索包括根据用户自然语言问题或新漏洞描述召回相似漏洞。使用 PostgreSQL + pgvector 可以在同一个数据库中同时完成两类需求，避免结构化数据和向量数据分散在不同系统中。

6.2 选用 LangGraph 的原因

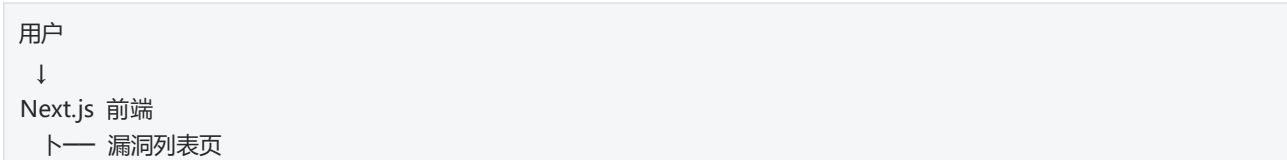
本项目的 AI 流程不是简单的一次问答，而是由文本预处理、结构化抽取、漏洞分类、风险评分、修复建议、向量生成、报告生成等多个步骤组成。LangGraph 可以将这些步骤显式建模为节点，便于记录中间结果、定位失败原因、重试失败节点和扩展新的分析逻辑。

6.3 开发策略

技术栈可以保持工程化，但开发顺序应保持稳妥。第一阶段先同步调用跑通后端、数据库和 AI 抽取；第二阶段接入 pgvector 完成 RAG；第三阶段再引入 Celery + Redis 进行异步化；第四阶段完善前端页面、看板和 Docker Compose。

7. 系统总体架构设计

系统采用前后端分离架构，整体分为前端展示层、后端服务层、AI 工作流层、数据存储层和异步任务层。

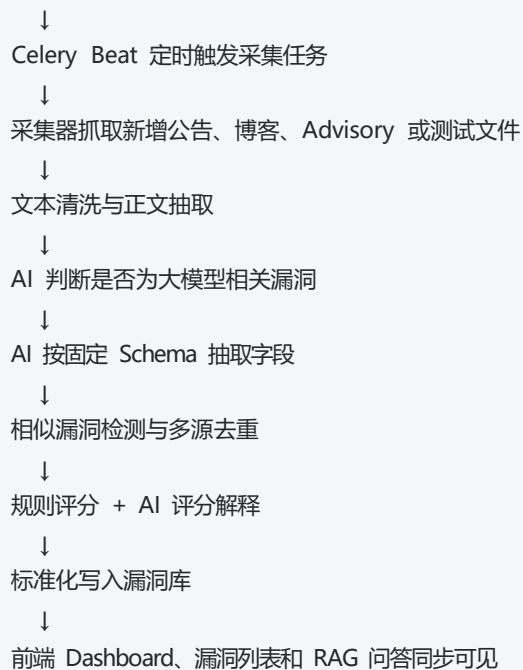




8. 核心业务流程设计

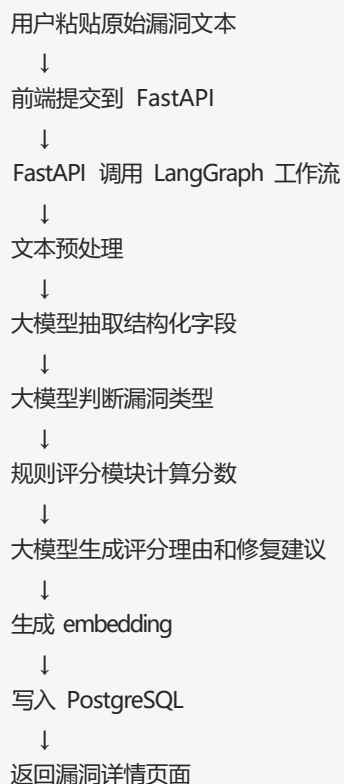
8.1 AI 漏洞动态采集与标准化入库流程

配置公开漏洞信息源



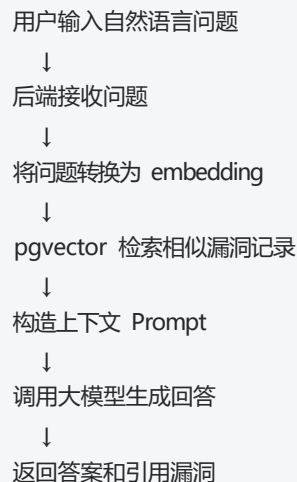
该流程直接对应课题要求中的实时动态搜集与固定格式上传。系统不是简单保存网页文本，而是将采集、AI 判断、结构化抽取、去重、评分和入库整合为可追踪的自动化链路。

8.2 漏洞录入与 AI 分析流程



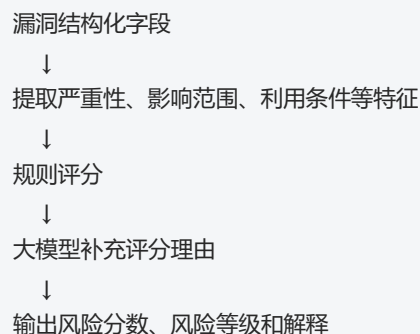
该流程是系统的核心主线，也是答辩时最适合演示的流程。用户只需要粘贴一段原始漏洞文本，系统即可完成结构化抽取、风险分析、修复建议生成和入库。

8.3 RAG 问答流程



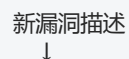
RAG 问答的关键在于回答必须基于漏洞库上下文。系统应在回答后展示参考漏洞记录，方便用户追溯答案来源，降低大模型幻觉带来的可信度问题。

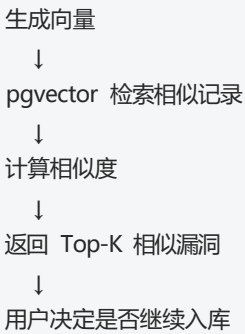
8.4 风险评分流程



风险评分不完全依赖大模型，而是采用“规则评分 + 大模型解释”的方式。这样既能提高分数稳定性，又能保留可读的分析理由。

8.5 相似漏洞检测流程





9. 功能模块详细设计

模块	子功能	设计说明
数据源管理模块	来源配置、启用禁用、采集周期	管理公开漏洞源、AI 框架公告、RSS/网页 URL 和本地测试源。
动态采集模块	定时采集、立即采集、正文解析、任务日志	利用 Celery Beat 周期性搜集 AI 漏洞候选文本，并记录采集状态。
漏洞管理模块	CRUD、搜索、筛选、排序	负责漏洞结构化数据的基础管理，是系统所有智能功能的基础。
AI 抽取模块	文本输入、字段抽取、人工确认	将非结构化漏洞文本转换为结构化字段。
风险分析模块	风险评分、等级映射、评分理由	采用规则计算分数，大模型生成解释。
RAG 问答模块	问题向量化、召回、上下文生成、回答	基于漏洞库回答用户问题，并展示引用记录。
相似检测模块	向量检索、Top-K 相似漏洞	录入新漏洞时提示可能重复记录。
报告生成模块	漏洞报告、修复建议、AI 分析结论	将漏洞详情整理为结构化报告。
数据看板模块	统计指标、分布图、Top 列表	用于整体了解漏洞库情况，并增强答辩展示效果。
任务管理模块	异步任务状态、失败原因、重试	用于后续接入 Celery，提升系统稳定性。

10. 数据库设计

数据库采用 PostgreSQL，向量字段使用 pgvector 扩展。核心数据表包括漏洞主表、标签表、漏洞标签关联表、AI 分析记录表、文档 Chunk 表、任务表、采集数据源表和原始采集文档表。

10.1 漏洞主表 vulnerabilities

字段名	类型	说明
id	UUID / Integer	漏洞 ID。
title	String	漏洞标题。
vuln_type	String	漏洞类型。

severity	String	风险等级。
score	Integer	风险评分。
affected_component	String	影响组件。
description	Text	漏洞描述。
attack_method	Text	攻击方式。
impact	Text	影响范围。
mitigation	Text	修复建议。
source	String	数据来源名称。
reference_url	String	参考链接。
source_url	String	采集到该漏洞的原始 URL。
collected_at	DateTime	动态采集时间。
confidence	Float	AI 判断和抽取结果置信度。
status	String	状态，例如待确认、未修复、已修复。
created_at / updated_at	DateTime	创建时间和更新时间。

10.2 标签表 tags 与关联表 vulnerability_tags

表名	字段	说明
tags	id, name	存储标签 ID 和标签名称。
vulnerability_tags	vulnerability_id, tag_id	维护漏洞与标签的多对多关系。

10.3 AI 分析记录表 analysis_records

字段名	类型	说明
id	UUID / Integer	分析记录 ID。
vulnerability_id	UUID / Integer	对应漏洞 ID。
summary	Text	AI 摘要。
risk_reason	Text	风险评分理由。
suggested_fix	Text	修复建议。
model_name	String	使用的模型名称。
prompt_version	String	Prompt 版本。
created_at	DateTime	生成时间。

10.4 文档 Chunk 表 document_chunks

字段名	类型	说明
id	UUID / Integer	Chunk ID。
vulnerability_id	UUID / Integer	对应漏洞 ID。
chunk_text	Text	用于向量检索的文本片段。
embedding	Vector	向量数据。
created_at	DateTime	创建时间。

10.5 任务表 tasks

字段名	类型	说明
id	UUID / Integer	任务 ID。
task_type	String	任务类型，例如 analyze、embed、crawl、report。
status	String	pending / running / success / failed。
input_data	JSON	输入数据。
output_data	JSON	输出数据。
error_message	Text	错误信息。
created_at / updated_at	DateTime	创建时间和更新时间。

10.6 采集数据源表 data_sources

字段名	类型	说明
id	UUID / Integer	数据源 ID。
name	String	数据源名称。
source_type	String	数据源类型，例如 rss、web、github、local_file。
url	String	数据源地址或本地文件路径。
enabled	Boolean	是否启用。
interval_minutes	Integer	采集周期，单位为分钟。
last_collected_at	DateTime	最近一次采集时间。
created_at / updated_at	DateTime	创建时间和更新时间。

10.7 原始采集文档表 collected_documents

字段名	类型	说明
id	UUID / Integer	原始文档 ID。
source_id	UUID / Integer	对应数据源 ID。
title	String	采集到的原始标题。
url	String	原始链接。
raw_text	Text	采集到的原始文本。
content_hash	String	内容哈希，用于去重。
is_ai_related	Boolean	AI 判断是否与大模型漏洞相关。
confidence	Float	AI 判断置信度。
status	String	pending / processed / ignored / failed。
vulnerability_id	UUID / Integer	标准化入库后关联的漏洞 ID。
collected_at	DateTime	采集时间。

11. 后端 API 设计

11.1 漏洞管理 API

方法	路径	功能
GET	/api/v1/vulnerabilities	获取漏洞列表，支持搜索、筛选、排序和分页。
GET	/api/v1/vulnerabilities/{id}	获取漏洞详情。
POST	/api/v1/vulnerabilities	新增漏洞。
PUT	/api/v1/vulnerabilities/{id}	修改漏洞。
DELETE	/api/v1/vulnerabilities/{id}	删除漏洞。

11.2 AI 分析 API

方法	路径	功能
POST	/api/v1/ai/extract	从原始文本中抽取漏洞字段。
POST	/api/v1/ai/analyze	对漏洞进行完整 AI 分析。
POST	/api/v1/ai/score	生成风险评分和评分理由。
POST	/api/v1/ai/report	生成漏洞报告。

11.3 RAG 问答 API

方法	路径	功能
POST	/api/v1/rag/ask	基于漏洞库问答。
POST	/api/v1/rag/search	语义检索相似漏洞。

11.4 动态采集 API

方法	路径	功能
GET	/api/v1/sources	获取采集数据源列表。
POST	/api/v1/sources	新增采集数据源。
PUT	/api/v1/sources/{id}	修改采集数据源。
DELETE	/api/v1/sources/{id}	删除采集数据源。
POST	/api/v1/collectors/run	手动触发一次采集任务。
GET	/api/v1/collectors/documents	查看原始采集文档和处理状态。
POST	/api/v1/collectors/documents/{id}/approve	人工确认并入库低置信度文档。

11.5 任务管理 API

方法	路径	功能
POST	/api/v1/tasks/collect	创建动态采集任务。
POST	/api/v1/tasks/analyze	创建异步分析任务。

GET	/api/v1/tasks/{id}	查询任务状态。
POST	/api/v1/tasks/{id}/retry	重试失败任务。

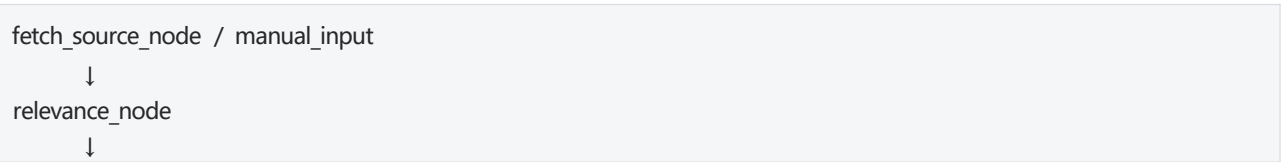
12. AI 工作流与 Prompt 设计

12.1 LangGraph 状态对象设计

```
class VulnerabilityAnalysisState:
    source_id: str
    source_url: str
    raw_text: str
    cleaned_text: str
    is_ai_related: bool
    confidence: float
    extracted_fields: dict
    vuln_type: str
    severity: str
    score: int
    risk_reason: str
    mitigation: str
    embedding_id: str
    report: str
    errors: list[str]
```

12.2 workflow 节点设计

节点	输入	输出
fetch_source_node	数据源配置	候选漏洞文本列表。
relevance_node	候选文本	是否为 AI 大模型相关漏洞、置信度和理由。
preprocess_node	原始文本	清洗后的文本。
extract_node	清洗文本	结构化漏洞字段。
classify_node	漏洞字段	漏洞类型、风险类别。
score_node	漏洞字段	风险分数、评分理由。
mitigation_node	漏洞字段	修复建议。
deduplicate_node	漏洞字段和向量	相似漏洞列表和去重决策。
embedding_node	漏洞描述	向量数据。
report_node	全部分析结果	漏洞报告。



```
preprocess_node
  ↓
extract_node
  ↓
classify_node
  ↓
score_node
  ↓
mitigation_node
  ↓
deduplicate_node
  ↓
embedding_node
  ↓
report_node
```

12.3 AI 漏洞相关性判断 Prompt

你是一个 AI 安全情报筛选助手。请判断下面的文本是否描述了 AI 大模型、LLM 应用、Agent、RAG、模型插件、模型供应链或模型服务相关的安全漏洞。

要求：

1. 只输出 JSON。
2. 不要因为文本中出现 AI 字样就直接判断为漏洞。
3. 如果只是普通新闻、产品介绍或非漏洞内容，请返回 false。
4. confidence 取值范围为 0 到 1。

输出字段：

- is_ai_vulnerability
- confidence
- related_area
- reason

候选文本：

{{candidate_text}}

12.4 结构化抽取 Prompt

你是一个 AI 应用安全分析助手。请从下面的漏洞描述中抽取结构化信息。

要求：

1. 只输出 JSON。
2. 不要编造原文中没有的信息。

3. 如果某个字段无法判断, 请填入 "unknown".
4. 风险等级只能从 ["低危", "中危", "高危", "严重"] 中选择。
5. 漏洞类型应尽量归纳为 AI 大模型应用安全相关类别。

需要抽取的字段:

- title
- vuln_type
- affected_component
- severity
- description
- attack_method
- impact
- mitigation
- tags

漏洞描述:

{{raw_text}}

12.5 风险评分解释 Prompt

你是一个安全风险评估助手。请根据给定漏洞信息, 解释该漏洞的风险评分理由。

注意:

1. 风险分数已经由规则系统计算得到。
2. 你的任务是解释评分原因, 而不是重新编造分数。
3. 请从攻击难度、影响范围、是否涉及敏感数据、是否容易修复等角度分析。

漏洞信息:

{{vulnerability_fields}}

规则评分:

{{score}}

请输出:

- risk_reason
- key_risk_factors
- suggested_priority

12.6 RAG 问答 Prompt

你是一个 AI 大模型漏洞库问答助手。请根据给定的漏洞库上下文回答用户问题。

要求:

- 1. 回答必须基于上下文。
- 2. 如果上下文中没有相关信息，请说明“钻漏洞库中未找到充分依据”。
- 3. 不要编造漏洞编号、来源或影响范围。
- 4. 回答最后列出参考的漏洞标题。

用户问题：
{{question}}

漏洞库上下文：
{{retrieved_context}}

13. 前端页面设计

13.1 页面结构

页面	功能
首页 / Dashboard	展示漏洞统计、风险分布、最近漏洞和高风险漏洞。
漏洞列表页	展示、搜索、筛选、排序漏洞。
漏洞详情页	查看漏洞完整信息、AI 分析结果、报告和相似漏洞。
新增漏洞页	手动新增漏洞。
AI 抽取页	粘贴原始文本，自动抽取漏洞字段并确认入库。
动态采集中心页	配置漏洞数据源、触发采集任务、查看采集状态和待确认文档。
RAG 问答页	与漏洞库进行自然语言问答，并展示引用记录。
任务中心页	查看异步任务状态、失败原因和重试入口。
设置页	配置模型 API、Prompt 版本等。

13.2 漏洞列表页设计

搜索框

风险等级筛选器

漏洞类型筛选器

影响组件筛选器

漏洞表格

分页组件

新增漏洞按钮

AI 导入按钮

表格字段：

标题 | 类型 | 风险等级 | 风险评分 | 影响组件 | 状态 | 创建时间 | 操作

13.3 AI 抽取页设计

左侧：原始漏洞文本输入框

右侧：AI 抽取结果预览

下方：确认入库按钮

用户操作：
粘贴文本 → 点击 钻AI 解析” → 查看结构化字段 → 手动修改不准确字段 → 点击 钻确认入库”

该页面设计强调“人工确认”。大模型输出不能完全无审核入库，用户应能在入库前修改字段，保证漏洞库质量。

13.4 动态采集中心页设计

页面区域：

1. 数据源列表：展示名称、类型、URL、启用状态、采集周期、最近采集时间。

2. 新增数据源表单：支持 RSS、网页 URL、本地测试文件等类型。

3. 立即采集按钮：手动触发 collect_source 任务。

4. 任务状态区：展示 pending、running、success、failed 状态和错误信息。

5. 待确认队列：展示 AI 低置信度或疑似重复的采集结果，支持人工确认入库或忽略。

该页面用于直观展示“实时动态搜集”的效果。答辩时可以配置一个测试数据源，点击立即采集后展示系统自动抓取候选文本、AI 判断相关性、抽取字段、去重并入库。

13.5 RAG 问答页设计

上方：问题输入框

中间：AI 回答区域

右侧：召回漏洞列表

下方：参考记录详情

示例：

问题：Prompt Injection 漏洞如何防护？

回答：Prompt Injection 通常通过用户输入、工具返回内容或外部文档内容注入恶意指令

参考漏洞：

1. Agent 工具调用中的 Prompt Injection 漏洞

2. RAG 外部文档注入漏洞

14. 风险评分算法设计

为了避免完全依赖大模型，系统采用规则评分。评分规则可以随着项目迭代不断调整， MVP 阶段使用可解释、简单稳定的规则即可。

条件	加分/扣分
严重等级为严重	+50
严重等级为高危	+40
严重等级为中危	+25
严重等级为低危	+10
可远程利用	+15
涉及敏感数据泄露	+15
存在公开 PoC	+10
影响核心组件	+10
修复方案未知	+10
需要高权限才能利用	-10
已有明确修复方案	-10

```
def calculate_risk_score (vu ln):
    score = 0

    severity_map = {
        "严重": 50,
        "高危": 40,
        "中危": 25,
        "低危": 10,
    }
    score += severity_map.get(vu ln.severity, 20)

    if vu ln.remote_exploitable:
        score += 15
    if vu ln.sensitive_data_involved:
        score += 15
    if vu ln.public_poc_available:
        score += 10
    if vu ln.core_component_affected:
        score += 10
    if not vu ln.fix_available:
        score += 10
    if vu ln.high_privilege_required:
        score -= 10
    if vu ln.fix_available:
        score -= 10

    return max(0, min(score, 100))
```

15. 测试方案设计

15.1 功能测试

测试对象	测试内容
漏洞管理	新增、删除、修改、查询是否正常。
动态采集	数据源是否能被触发采集，新增漏洞是否能进入待处理队列或漏洞库。
AI 相关性判断	采集到的文本是否能正确判断为 AI 漏洞或非相关内容。
AI 抽取	输入漏洞文本后是否能输出结构化字段。
风险评分	不同漏洞是否能得到合理分数和等级。
RAG 问答	用户问题是否能召回相关漏洞，并基于上下文回答。
相似检测	相似漏洞是否能被召回，重复录入是否有提示。
报告生成	是否能生成完整漏洞报告。
数据看板	统计结果是否与数据库一致。

15.2 典型测试用例

编号	输入/操作	预期结果
TC-01	配置本地测试源或 RSS 源并点击立即采集。	系统创建采集任务，抓取候选文本，AI 判断相关性，并将高置信度 AI 漏洞标准化入库。
TC-02	采集源中包含一条普通 AI 新闻和一条 RAG 漏洞描述。	普通新闻被标记为 ignored，RAG 漏洞进入结构化抽取流程。
TC-03	粘贴 Agent Prompt Injection 漏洞文本并点击 AI 解析。	输出漏洞类型为 Prompt Injection，风险等级为高危，给出攻击方式和修复建议。
TC-02	录入一条 RAG 知识库泄露相关漏洞。	系统生成 RAG / Data Leakage 标签，并在问答中可被召回。
TC-03	问题输入 “有哪些漏洞可能导致知识库内容泄露？力	系统召回 RAG 相关漏洞并总结知识库越权访问、检索权限控制不足等风险。
TC-04	输入与已有 Prompt Injection 漏洞相似的新漏洞描述。	系统返回相似漏洞 Top-3，并提示可能重复录入。
TC-05	修改漏洞状态为已修复并保存。	列表页与详情页状态同步更新。
TC-06	删除一条漏洞记录。	记录从列表消失，关联 Chunk 和标签关系正确处理。

16. 风险分析与解决方案

风险	表现	解决方案
----	----	------

大模型输出不稳定	JSON 格式错误、字段缺失、内容不准确。	使用 Pydantic 校验，失败后重试或提示人工修改。
项目范围过大	前端、后端、AI、数据库、任务队列都要实现。	先完成 MVP，再扩展高级功能。
RAG 效果不理想	召回结果不准确，回答不够贴合。	优化 Chunk 设计，增加关键词过滤和风险类型筛选。
pgvector 配置复杂	数据库扩展安装或向量维度配置出错。	使用 Docker Compose 固定环境。
Celery 增加复杂度	任务状态、重试和调试成本增加。	前期同步调用，后期再异步化。
数据来源不足	缺少真实 AI 漏洞样例。	先构造本地测试源，再逐步接入公开安全公告、GitHub Advisory 和论文摘要。
动态采集不稳定	网页结构变化、网络失败、反爬限制导致采集失败。	优先支持 RSS、API 和本地测试源；采集失败写入任务日志并允许重试。
采集内容噪声较多	普通新闻或非 AI 漏洞被采集进来。	采用关键词初筛+ AI 相关性判断 + 人工审核队列。
AI 分析可能幻觉	编造不存在的信息。	Prompt 中要求不确定则

		unknown，并保留人工确认流程。
--	--	--------------------

17. 预期成果与答辩展示方案

17.1 预期成果

- 1. 一个可运行的 Web 系统。
- 2. 一个包含若干 AI 大模型安全漏洞样例的漏洞库。
- 3. 支持从配置的数据源实时/定时动态搜集 AI 漏洞候选文本。
- 4. 支持 AI 判断候选文本是否属于大模型相关漏洞，并按固定格式标准化入库。
- 5. 支持漏洞增删改查、搜索筛选和详情查看。
- 6. 支持大模型自动抽取漏洞字段。
- 7. 支持风险评分和评分理由生成。
- 8. 支持基于漏洞库的 RAG 问答。
- 9. 支持漏洞分析报告生成。
- 10. 支持基础数据可视化看板。
- 11. 提供项目报告、系统架构图、功能截图和演示视频。

17.2 答辩展示主线

答辩演示建议围绕一条完整业务链路展开，而不是零散展示按钮。推荐演示流程如下：



18. 项目总结定位

本项目以 AI 大模型应用安全为背景，将传统漏洞库管理、大模型文本分析、向量检索和前后端工程系统结合起来，构建了一个面向 AI 安全场景的漏洞情报管理平台。

实时动态搜集 AI 漏洞

- + 大模型相关性判断
- + 标准化字段入库
- + 传统漏洞 CRUD
- + 大模型结构化抽取
- + 风险评分与解释
- + pgvector 语义检索
- + RAG 智能问答
- + LangGraph 工作流编排
- + 前后端分离工程实现

最终可以在课程报告中概括为：本系统围绕 AI 大模型应用安全中的漏洞情报管理问题，设计并实现了一个集动态漏洞采集、AI 相关性判断、标准化入库、漏洞管理、风险评估、智能检索问答和报告生成于一体的安全分析平台。系统通过 LangGraph 编排大模型分析流程，使用 PostgreSQL 与 pgvector 统一管理结构化漏洞数据和语义向量数据，并通过 Next.js 与 FastAPI 实现可交互的前后端系统，具有较好的实用性、扩展性和课程展示价值。

附录 A：示例测试数据

样例 1：Prompt Injection 漏洞

某 LLM Agent 应用将外部网页内容直接拼接进系统 Prompt 中，攻击者可以在网页中隐藏恶意指令，诱导模型泄露系统提示词或执行未授权工具调用。

期望输出：

- 漏洞类型：Prompt Injection
- 风险等级：高危
- 影响组件：LLM Agent / Web Retriever
- 影响范围：系统提示词泄露、越权工具调用
- 修复建议：隔离外部内容，增加指令边界，限制工具权限

样例 2：RAG 知识库泄露漏洞

某企业知识库问答系统在检索阶段未根据用户身份过滤文档权限，普通用户可以通过构造问题召回本不应访问的内部文档片段。

期望输出：

- 漏洞类型：RAG Data Leakage
- 风险等级：高危
- 影响组件：RAG Retriever / Document Store

- 影响范围：内部知识库内容泄露
- 修复建议：检索阶段增加权限过滤，对文档 Chunk 绑定访问控制标签

附录 B：项目目录结构建议

```
llm-vulnhub/  
├── backend/  
│   ├── app/  
│   │   ├── api/  
│   │   │   ├── vulnerabilities.py  
│   │   │   ├── sources.py  
│   │   │   ├── collectors.py  
│   │   │   ├── aipy  
│   │   │   ├── rag.py  
│   │   │   └── tasks.py  
│   │   ├── core/  
│   │   │   ├── config.py  
│   │   │   └── security.py  
│   │   ├── db/  
│   │   │   ├── session.py  
│   │   │   └── models.py  
│   │   ├── schemas/  
│   │   ├── services/  
│   │   │   ├── vulnerability_service.py  
│   │   │   ├── collector_service.py  
│   │   │   ├── llm_service.py  
│   │   │   ├── rag_service.py  
│   │   │   └── scoring_service.py  
│   │   ├── workflows/  
│   │   │   └── vuln_analysis_graph.py  
│   │   └── main.py  
│   ├── alembic/  
│   └── requirements.txt  
├── frontend/  
│   ├── app/  
│   │   ├── dashboard/  
│   │   │   ├── vulnerabilities/  
│   │   │   ├── ai-extract/  
│   │   │   ├── collectors/  
│   │   │   └── rag-chat/  
│   │   ├── components/  
│   │   └── package.json  
├── docker-compose.yml  
└── README.md
```